Fine-tuning LLMs and VLMs with open-source libraries

Sergio Paniego Blanco (@sergiopaniego)
Machine Learning Engineer @ Hugging Face

Hugging Face

Hi, I’m Sergio 👋

- Machine Learning Engineer @ Hugging Face 🤗
  - Post-Training
  - RL Envs
- PhD in AI @ URJC

Slides: https://github.com/sergiopaniego/talks

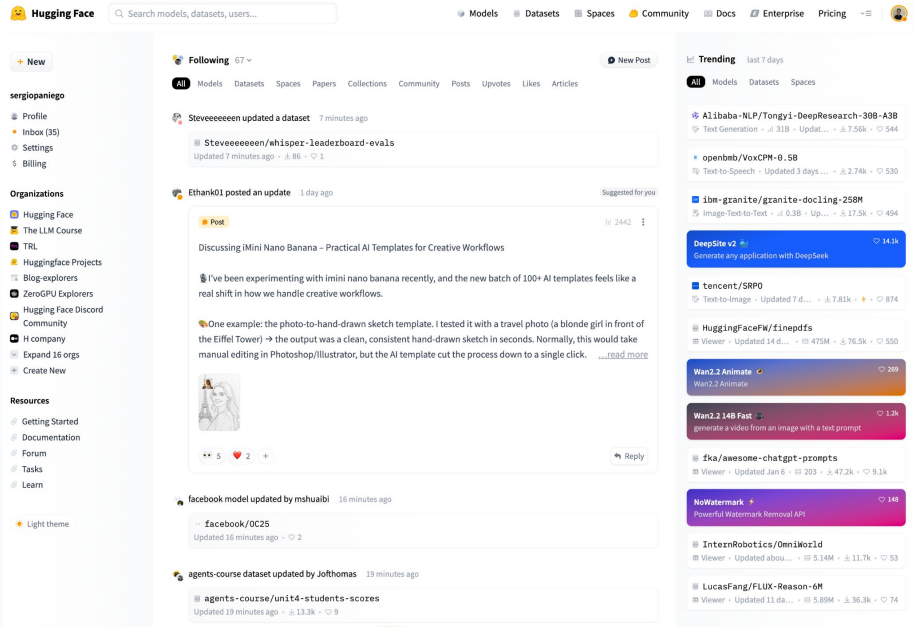
Quick intro to Hugging Face 🤗

AI open platform for building, sharing and deploying AI (HF Hub):

- Models, datasets, apps…
- Posts, blogs, courses, papers, leaderboards…

2.6M
public models

800K+
public datasets

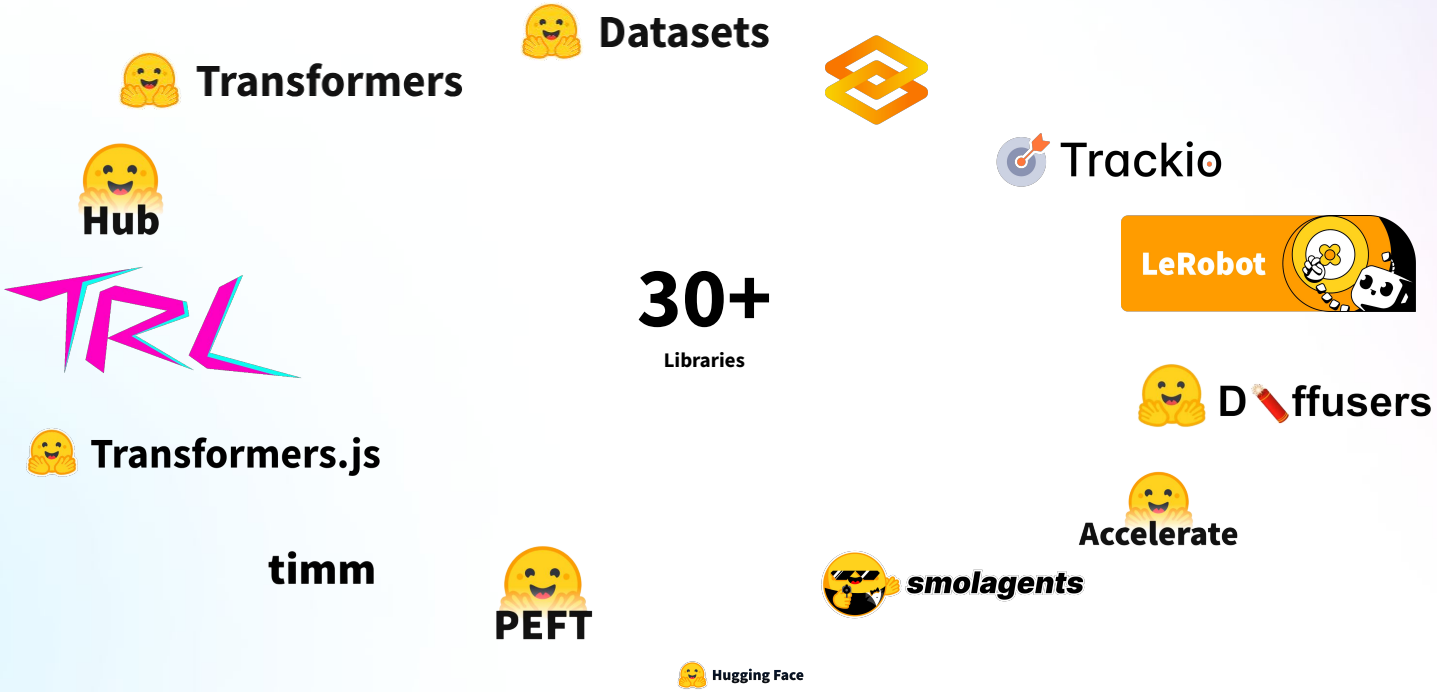


1M+
daily downloads

1M+
daily visitors

Quick intro to Hugging Face

Set of open source projects to empower AI development and advancement

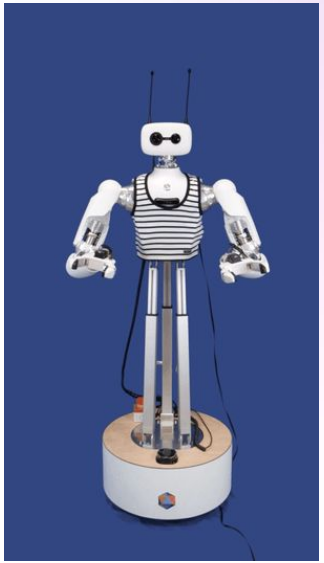


Quick intro to Hugging Face

We also do robotics!

LeRobot

POLLEN ROBOTICS



Quick intro to Hugging Face

Find model → Customize → Deploy

 **Transformers** TRL vLLM

From fine-tuning to TRL (fine tuning+alignment)

I fine-tuned the model and…

- 💬 It can’t chat naturally
- 🧠 It can’t reason or explain
- 🌀 It gives incoherent answers
- ⚠️ It’s not aligned with human preferences (unhelpful or unsafe)
- 🚫It doesn’t follow instructions
- …

From fine-tuning to TRL (fine tuning+alignment)

Classic fine-tuning adapts the model to **your dataset**, not necessarily to **your intentions.**

- Dataset (X → Y) → Pretrained model → Fine-tuned model
- Optimizes likelihood: predicts correctly based on data
- **Doesn't guarantee alignment** with human preferences

We may need something stronger (TRL), tools that can teach the model, not just fit it (TRL).

Data→ Model → Fine-tuned model

💭**(Still says weird stuff)**

Hugging Face

Where does TRL fit in the model lifecycle?

Pre-training: teaches general capacities such as language use, broad reasoning, and world knowledge
Mid-training: imparts domain knowledge (code, medial, databases, internal docs…)
Post-training/Alignment (TRL **):** learns to behave as we want (instruction following, reasoning, agentic behaviors**…)**

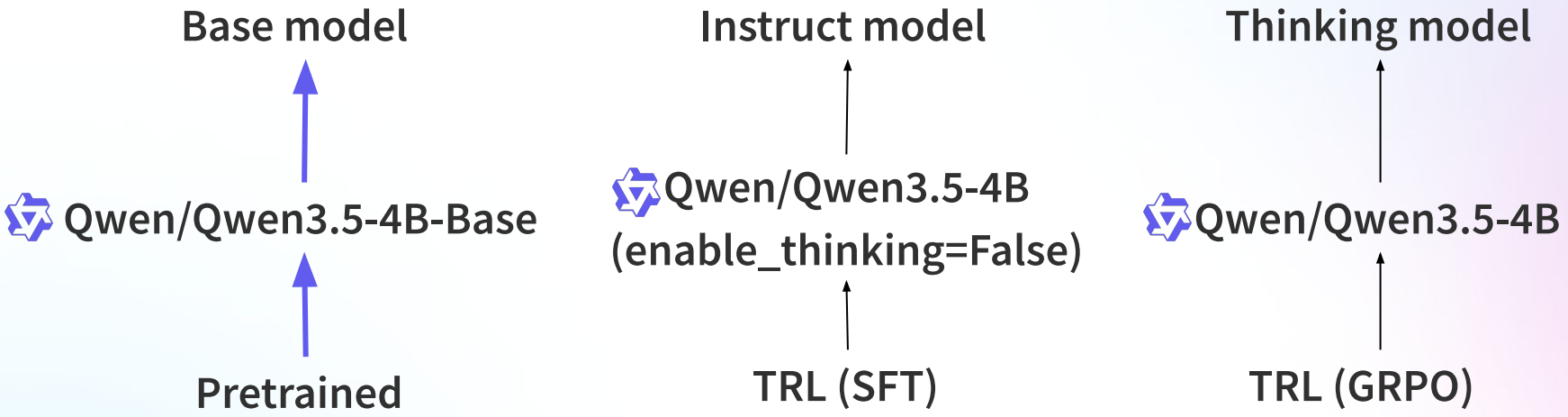
TRL operates in post-training→ where we optimize behavior, not just likelihood

If fine-tuning is not enough…

What exactly are we optimizing?

Stage	What is optimized?
Pretraining	Likelihood
Fine-tuning	Conditional likelihood
Alignment	Human preferences
RL	Behavior

From token prediction to agentic intelligence



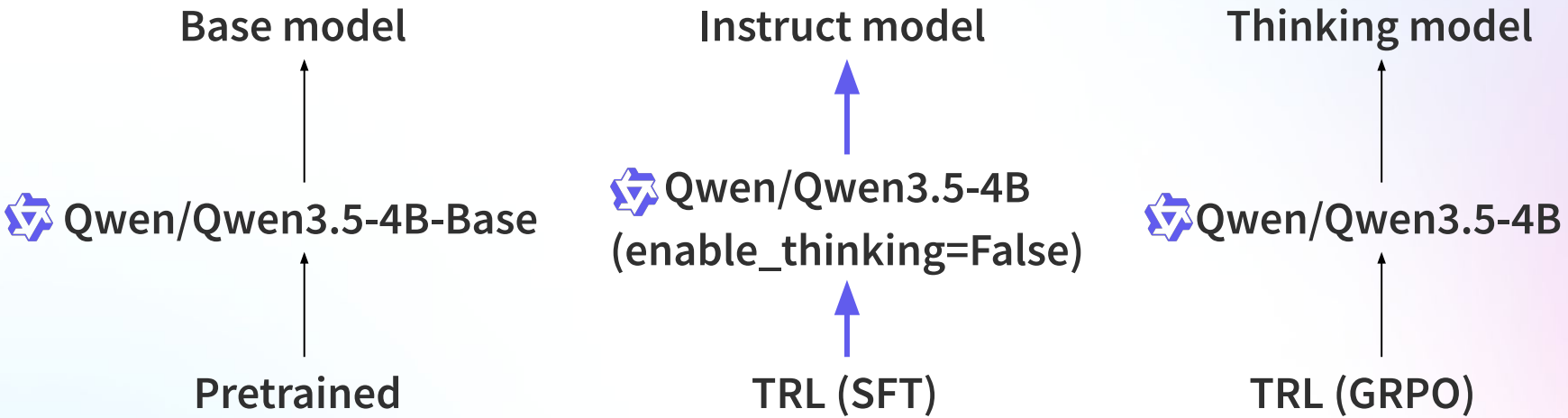
Base: predicts next token

Raw text, high quantity of data, masked language modeling, industrial compute

"Translate this sentence → Translate this sentence into…"

Optimizes: maximize P(next_token | context)

From token prediction to agentic intelligence



Instruct: follows instructions

Structures responses, high quality data, reduced compute

“<|user|>Translate this sentence</s>

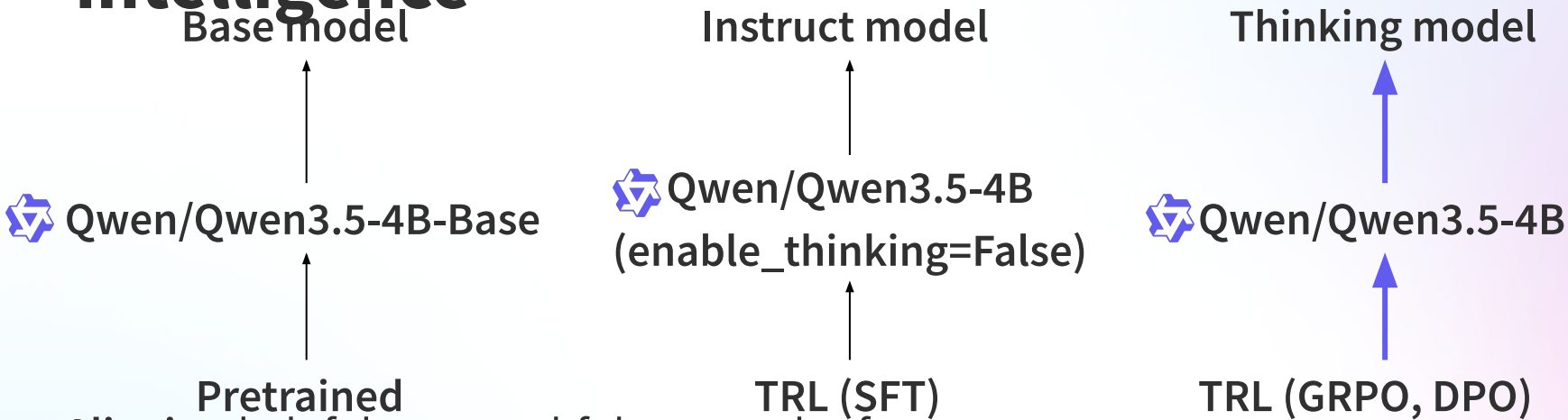
<|assistant|>Sure! Here’s the translation</s>

Example dataset

Optimizes: maximize P(response | instruction)

Hugging Face

From token prediction to agentic intelligence



Aligning: helpfulness, truthfulness, and safety

Preferences, high quality, preference optim, consumer compute

"<|user|>How many helicopters can a human eat?</s>

<|assistant|>Human cannot eat helicopters</s>

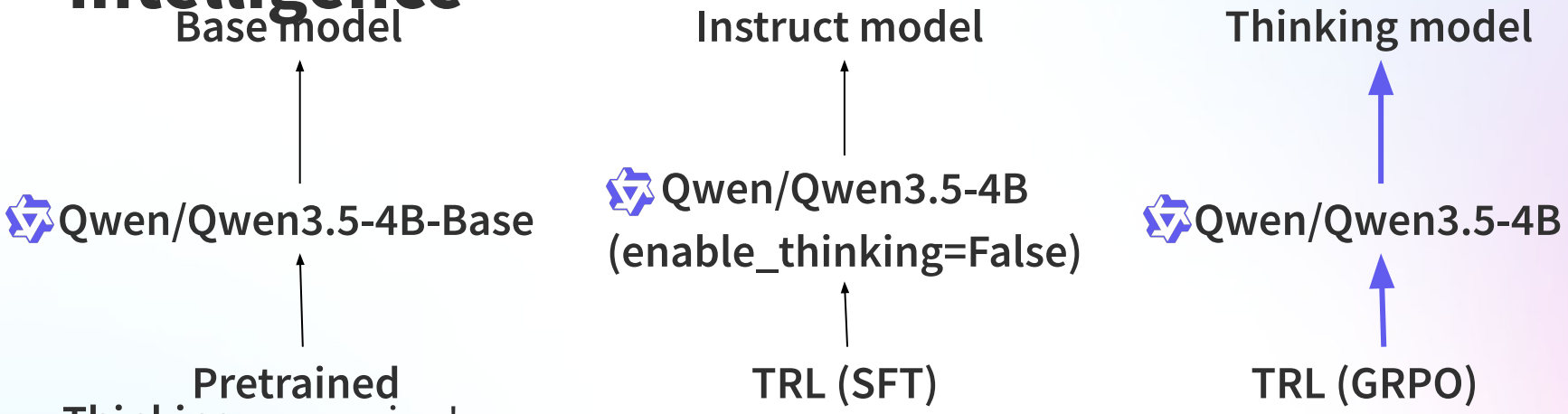
<|assistant|>That's not possible</s>

Example dataset

Optimizes: prefer chosen response over rejected response

Hugging Face

From token prediction to agentic intelligence



Thinking: reasoning!

"<|user|>How many helicopters can a human eat?</s>
<|assistant|>
<think>Tricky question. Let's break it down…</think>
That's not possible! I found that…</s>

Example dataset

Optimizes: maximize reward for reasoning trajectory

Hugging Face

From token prediction to agentic intelligence

From reasoning to Acting: reasoning is internal optimization.
With GRPO, we reward *how* the model thinks → Can we reward what it *does*?

From:

Prompt → <think> … </think> → Answer → Reward

To:

State → Action → Environment → Reward → New State → …

More on this later → OpenEnv!

TRL: Fine-tuning, alignment & reasoning

Library that offers specialized trainers (SFT, GRPO, PPO, DPO…)

- Bridges the gap between **instruction following** and **full alignment pipelines**
- Supports: **accelerate, PEFT, kernels, vLLM…**
- **Multimodal** models support
- **Focus: post-training → SFT, alignment (DPO), reasoning (GRPO)**

Hugging Face

Trainer vs TRL

TRL trainers are **built on top of transformers Trainer**, enhancing its functionality

Each trainer **abstracts a full training method**, so you don’t need to reimplement everything from scratch

Gives ML engineers **ready-to-use recipes** for SFT, GRPO, PPO, DPO…

Focus: practicality, reproducibility, and reduced boilerplate

```
from trl import SFTTrainer
from datasets import load_dataset

trainer = SFTTrainer(
    model="Qwen/Qwen3-0.6B",
    train_dataset=load_dataset("trl-lib/Capybara", split="train"),
)
trainer.train()
```

SFT and GRPO

We'll focus here on two trainers, but there are more

Supervised Fine-Tuning (SFT)

- Teach the model to follow instructions
- Uses human-labeled data (prompt → response)
- Improves helpfulness

Group Relative Policy Optimization

- Train with online RL
- Uses reward functions
- Improves alignment and reasoning

TRL taxonomy

Online methods

(learn from feedback) Model generates responses and learns from reward signals during training

- GRPO⚡
- RLOO⚡
- OnlineDPO⚡
- NashMD⚡
- XPO⚡
- PPO

Reward modeling

Training a model to judge responses

- PRM
- Reward

⚡= vLLM support

Offline methods

(data-driven) Model learns from human preference datasets or supervised signals

- SFT
- DPO
- ORPO
- BCO
- CPO
- KTO

Knowledge distillation

Transferring knowledge from a stronger model to a smaller one

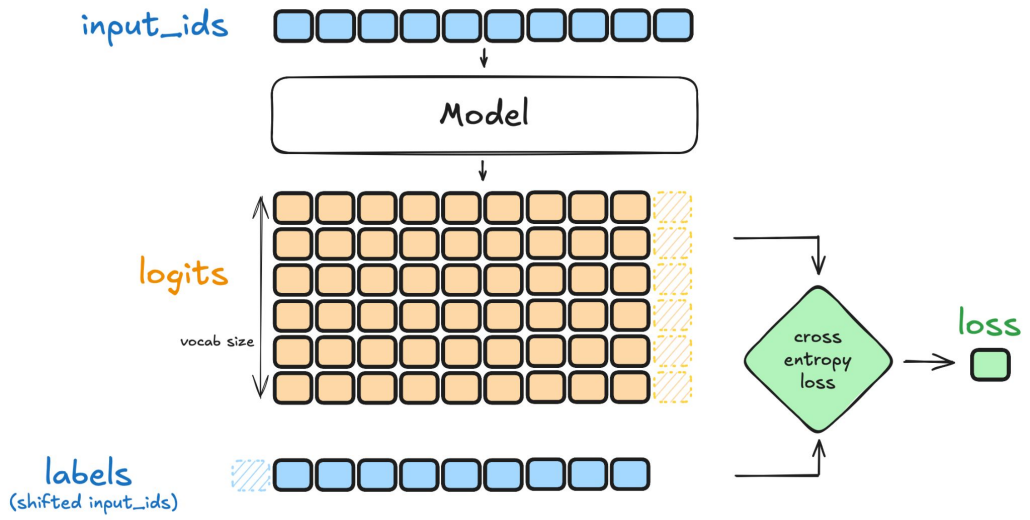
- GKD

Hugging Face

Supervised Fine-Tuning

SFT is the simplest and most commonly used method for adapting an LLM to a target dataset

- Goal: minimize negative log-likelihood of a sequence
- Simple, stable, and super effective



Supervised Fine-Tuning

SFTTrainer supports both standard and conversational dataset formats

```
# Standard
language_modeling_example = {"text": "The sky is blue."}

# Conversational
messages = [
    {"role": "user", "content": "Hello, how are you?"},
    {"role": "assistant", "content": "I'm doing great. How can I help you today?"},
    {"role": "user", "content": "I'd like to show off how chat templating works!"},
]
```

SFT is easy to instantiate and train

Only needs a model (even just the name!) and a dataset from the Hub to train

```python
from trl import SFTTrainer
from datasets import load_dataset

trainer = SFTTrainer(
    model="Qwen/Qwen3-0.6B",
    train_dataset=load_dataset("trl-lib/Capybara", split="train"),
)
trainer.train()
```

SFTTrainer and SFTConfig

```python
from trl import SFTTrainer, SFTConfig
from datasets import load_dataset

trainer = SFTTrainer(
    model="Qwen/Qwen3-0.6B",
    train_dataset=load_dataset("trl-lib/Capybara", split="train"),
    args = SFTConfig(
      # Training schedule / optimization
      per_device_train_batch_size = 2,      # Batch size per GPU
      gradient_accumulation_steps = 8,      # Gradients are accumulated over multiple steps → effective batch size = 2 * 8 = 16
      num_train_epochs = 1,                # Number of full dataset passes
      learning_rate = 2e-4,                 # Learning rate for the optimizer
      optim = "paged_adamw_8bit",           # Optimizer

      # Logging / reporting
      logging_steps=1,                      # Log training metrics every N steps
      report_to="trackio",                  # Experiment tracking tool

      # Hub integration
      push_to_hub=True,                     # Automatically push the trained model to the Hugging Face Hub
      ...
    )
    peft_config=LoraConfig(),
    # ...
)
trainer.train()
```

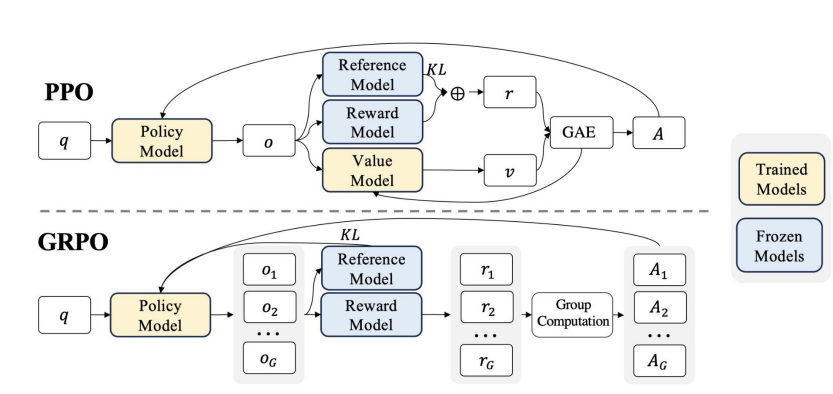
Group Relative Policy Optimization

GRPO was described in *DeepSeekMath* paper

Variant of Proximal Policy Optimization (PPO), that enhances maths reasoning capabilities while optimizing memory usage.

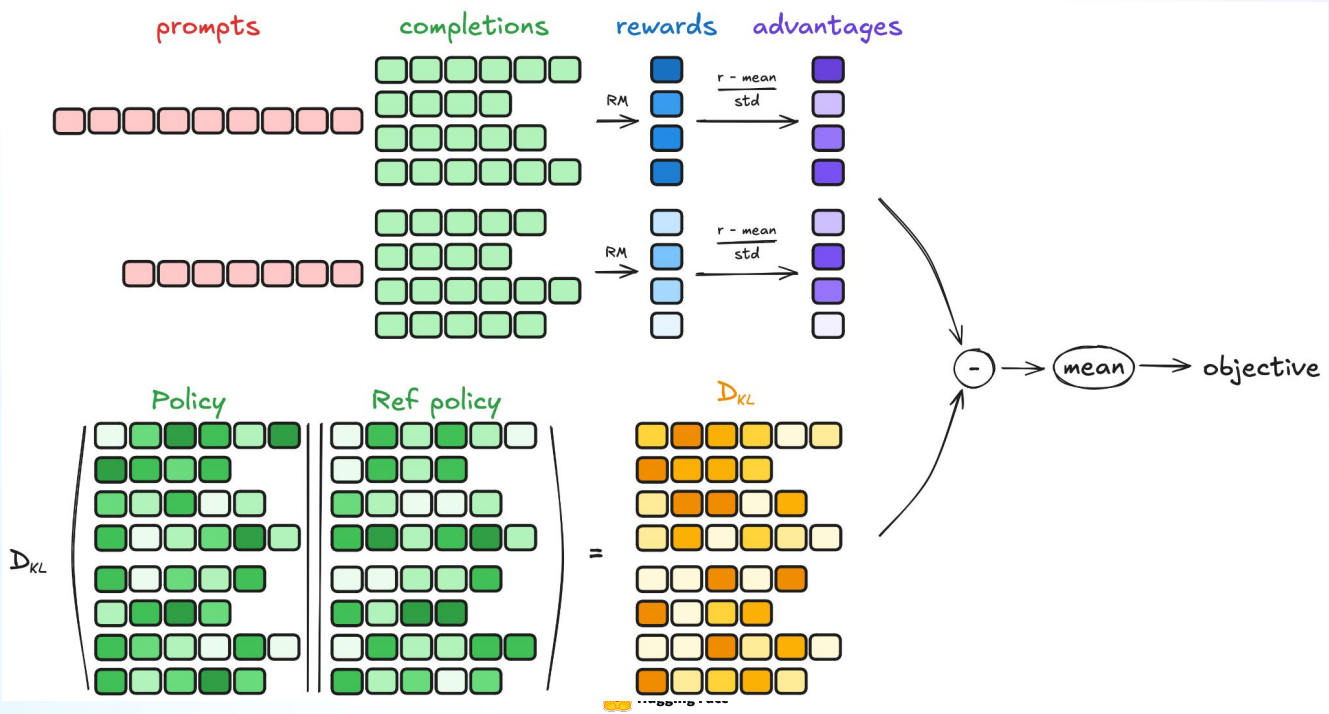
- Online method
- Optimized behavior using reward functions

Successor of PPO: more stable and memory efficient



GRPO

Steps: generating completions, computing advantage, computing the loss



GRPOTrainer and GRPOConfig

```python
from trl import GRPOTrainer, GRPOConfig
import re

def format_reward_func(completions, **kwargs):
    """Reward function that checks if the completion has a specific format."""
    pattern = r"^<think>.*?</think><answer>.*?</answer>$"
    completion_contents = [completion[0]["content"] for completion in completions]
    matches = [re.match(pattern, content) for content in completion_contents]
    return [1.0 if match else 0.0 for match in matches]

trainer = GRPOTrainer(
    model="Qwen/Qwen2-0.5B-Instruct",
    reward_funcs=[format_reward_func, ...],
    args=GRPOConfig(...),
    train_dataset=load_dataset("trl-lib/tldr", split="train"),
    # ...
)
trainer.train()
```

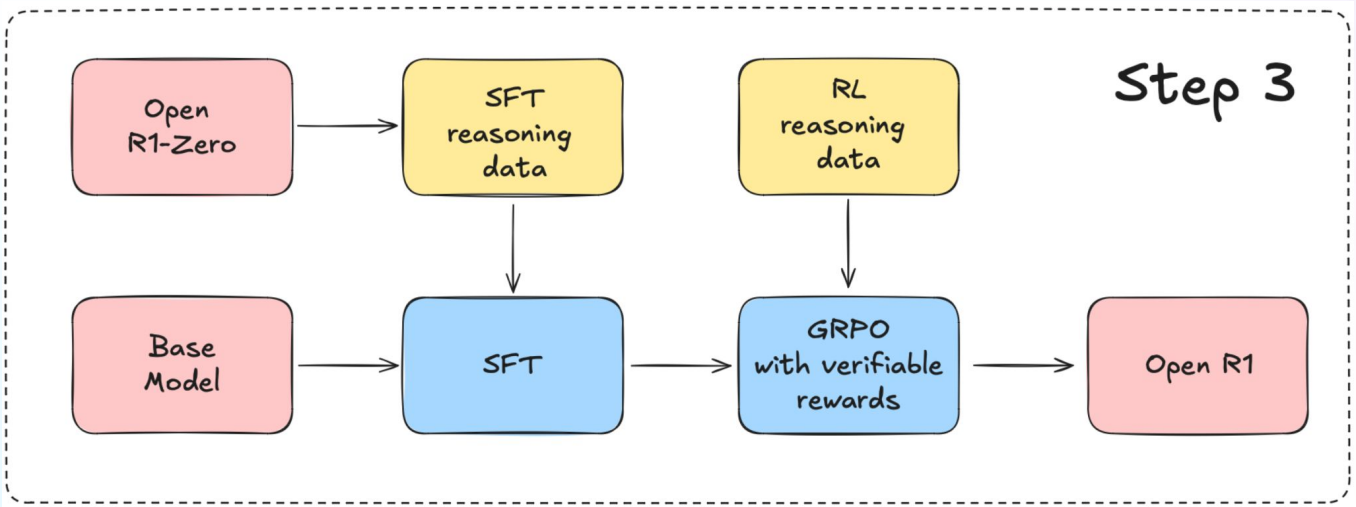
GRPOConfig detail

```python
from trl import GRPOConfig

# Configure training arguments using GRPOConfig
training_args = GRPOConfig(
    # ... Similar to SFTConfig

    # Parameters that control de data preprocessing
    max_completion_length=512,  # default: 256
    num_generations=8,  # default: 8
    max_prompt_length=512,  # default: 512

)
```

OpenR1

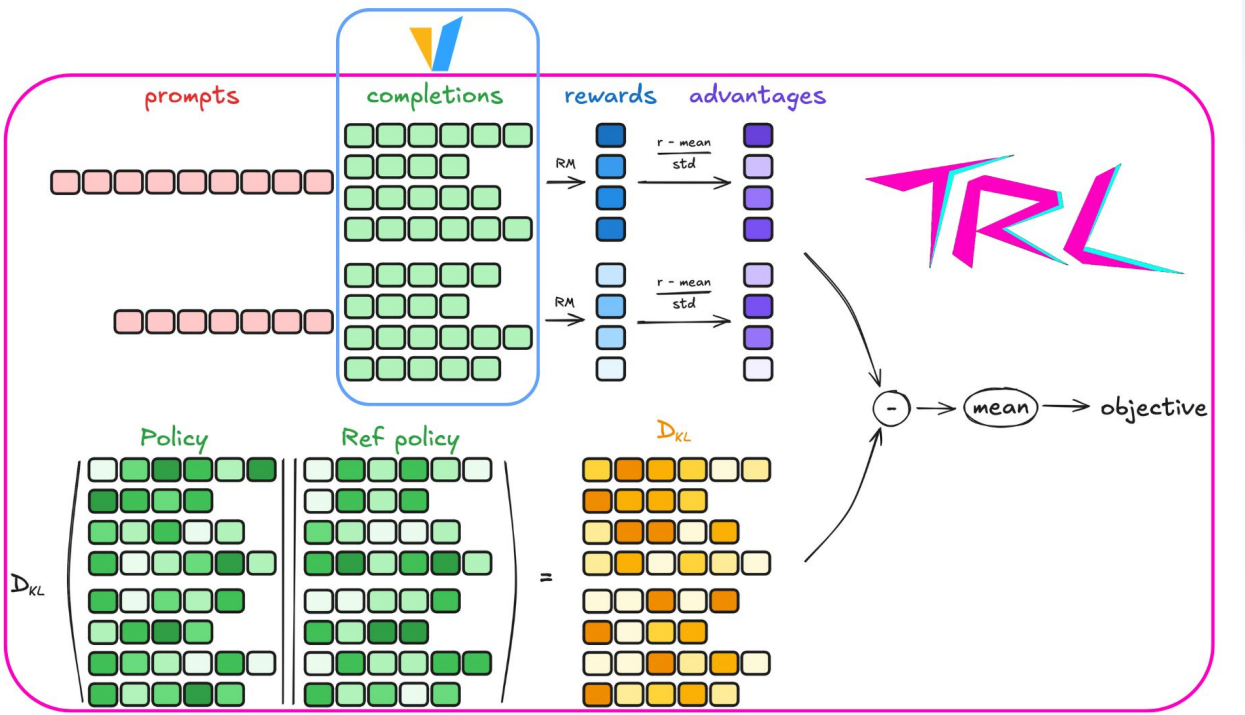
Initiative by HF to replicate and extend the techniques behind DeepSeek-R1 🐳

This model was built using SFT+GRPO!



Can we improve online training?

Some trainers work online → the model generates completions during training to compute the reward signal



Can we improve online training?

🚧 **Bottleneck**! Generation is slow and inefficient

⚡ **vLLM** solves this problem and is supported by TRL

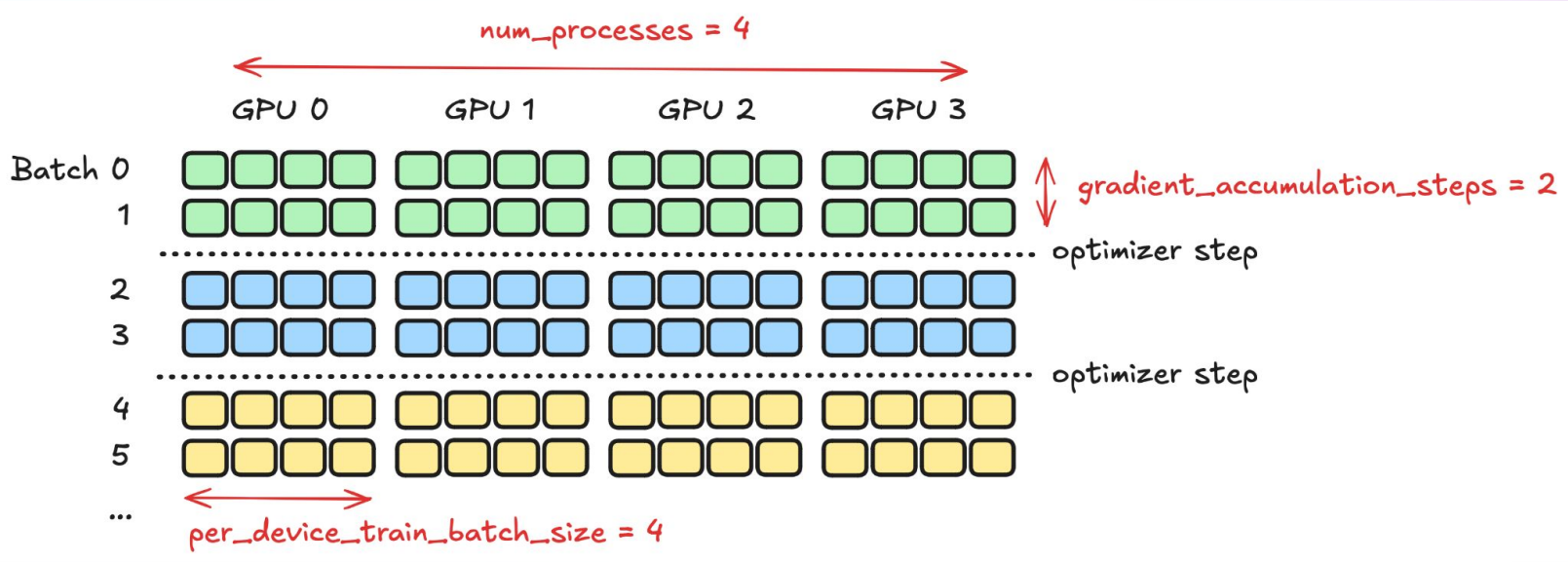
You can call it from the **CLI**

```
CUDA_VISIBLE_DEVICES=0,1,2,3 trl vllm-serve --model Qwen/Qwen2.5-7B --tensor-parallel-size 2 --data-parallel-size 2
```

```
from trl import GRPOConfig

training_args = GRPOConfig(
    ...,
    use_vllm=True,
    vllm_mode="server",  # default value, can be omitted. Can also be "colocate"
)
```

But can we scale training?

TRL also integrates seamlessly with accelerate and DeepSpeed (model sharding, zero redundancy optimized, mixed precision training, offloading…) deepspeed



But can we scale training?

In the config we specify num of machines, num of processes…

```
accelerate launch --config_file examples/accelerate_configs/deepspeed_zero2.yaml train.py
```

But can we scale training?

Train long context via **context parallelism** (accelerate): by splitting the sequence across multiple GPUs

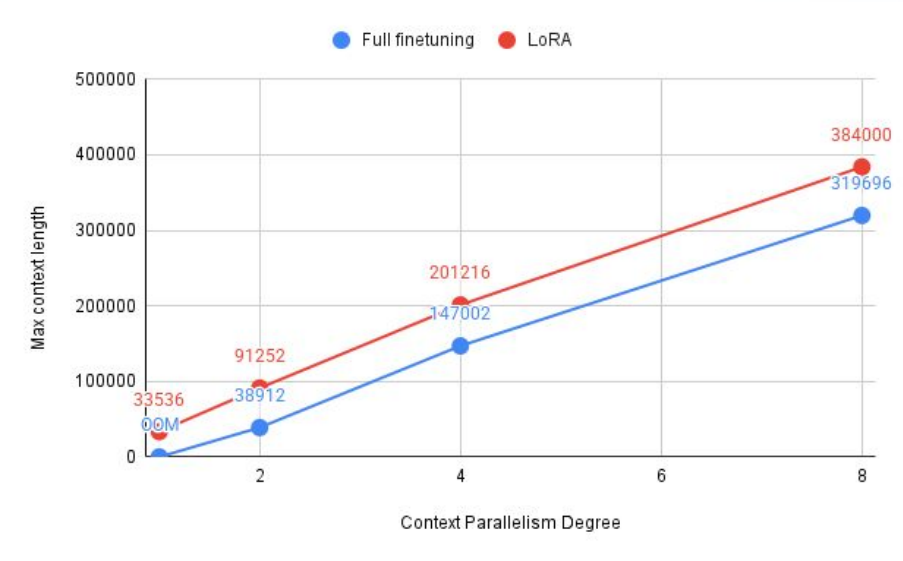
Even **ND-parallelism** (N model replicas, N devices, tensor parallelism, context parallelism)

```
accelerate launch --config_file context_parallel_2gpu.yaml train.py
```

```
from trl import SFTConfig

training_args = SFTConfig(
    # required
    pad_to_multiple_of=4,           # ensures divisibility by cp_size * 2
    # to get the most out of CP
    max_length=400000,              # long sequence length
    packing=True,                   # use packing to reduce padding
    use_liger_kernel=True,          # compatible with CP
    gradient_checkpointing=False,   # The activation_checkpointing in FSDP config and
the gradient_checkpointing in training arg can't be set to True simultaneously
    per_device_train_batch_size=1,
    ...
)
```

But can we scale training?

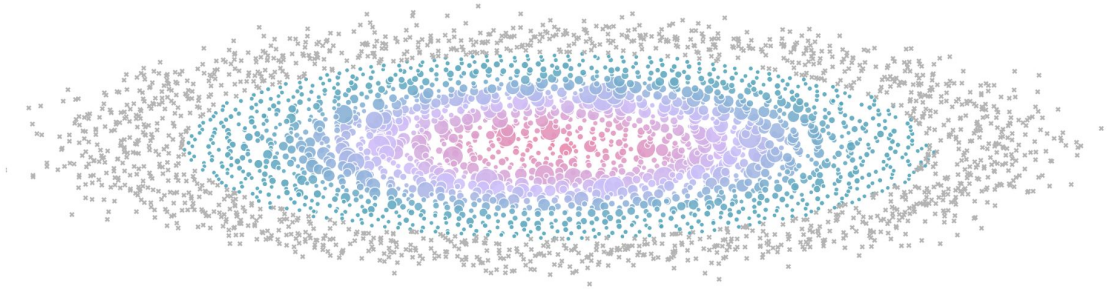
Context Parallelism (CP) with *Qwen/Qwen3-8B* scales to over **300k tokens in 8GPUs** 🙌



But can we scale training?

nanotron / ultrascale-playbook

The Ultra-Scale Playbook: Training LLMs on GPU Clusters



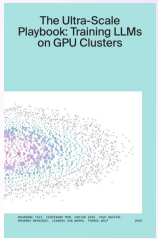
We ran over 4,000 scaling experiments on up to 512 GPUs and measured throughput (size of markers) and GPU utilization (color of markers). Note that both are normalized per model size in this visualization.

ORDER BOOK GET PDF

AUTHORS
Nouamane Tazi, Ferdinand Mom, Haojun Zhao, Phuc Nguyen, Mohamed Mekkouri, Leandro Werra, Thomas Wolf

AFFILIATION
Hugging Face

PUBLISHED
Feb 19, 2025



Hugging Face

Optimizing all the things!

Transformers already provides us with some optimizing strategies like gradient checkpointing and we also have LoRA/QLora. In addition, TRL has:

- Truncation
- Packing
- Padding-free
- Padding sequences to a multiple
- Liger kernels 

- Activation offloading

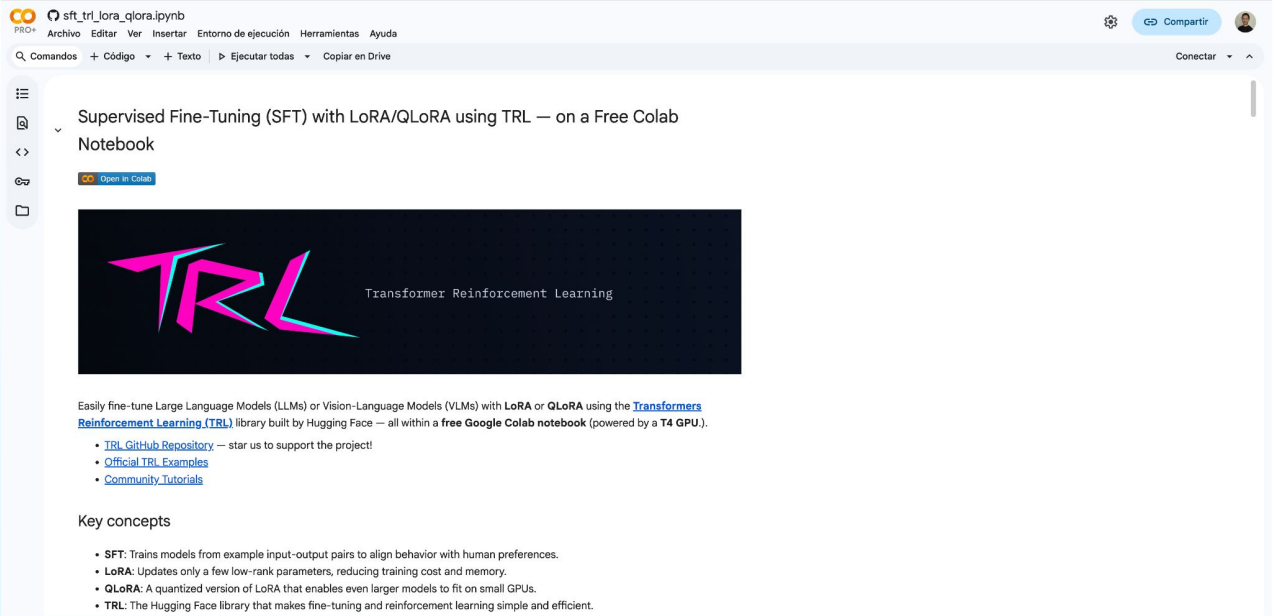
We developed optimized notebooks for training in free Colab.
For even more optimized training, we also have Unsloth 

Optimizing is easy

```python
from trl import SFTConfig

training_args = SFTConfig(
    max_length=1024,                        # (Truncation) Maximum input sequence length
    packing=True                            # Packing
    padding_free=True, model_init_kwargs={"attn_implementation": "flash_attention_2"}, # Padding-free
    pad_to_multiple_of=2048                 # Padding sequences to a multiple
    use_liger_kernel=True,                  # Enable Liger kernel optimizations for faster training
    activation_offloading=True,             # Offload activations to CPU to reduce GPU memory usage

    gradient_checkpointing=True,            # Save memory by re-computing activations during backpropagation
)
```

Optimizing for GPU poors

We developed [notebooks](#) for training (14B) models in free Colab!



HF jobs + TRL

Models can be trained using HF's infra via HF jobs

- **trl-jobs** cli package
- **hf jobs** (cli or python)

```
# pip install trl-jobs
trl-jobs sft --model_name Qwen/Qwen3-0.6B --dataset_name trl-lib/Capybara
```

```
hf jobs uv run \
    --flavor a100-large \
    --with trl \
    --secrets HF_TOKEN \
    train.py
```

```
from huggingface_hub import run_uv_job

run_uv_job(
    "train.py",
    dependencies=["trl"],
    flavor="a100-large",
    secrets={"HF_TOKEN": "hf_..."},
)
```

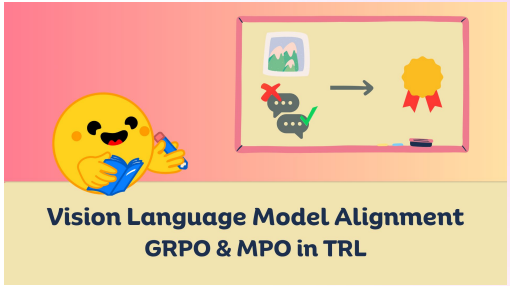
Multimodality

TRL supports **VLMs** in many trainers, including examples and recipes.

- SFT
- DPO (Direct Preference Optimization)

- MPO (Mixed Preference Optimization)
- GRPO (Group Relative Policy Optimization)
- GSPO (Group Sequence Policy Optimization)

- RLOO (Reinforce Leave One Out)
- Online DPO



Multimodality

Native SFT support for VLMs

You can still use your own collator

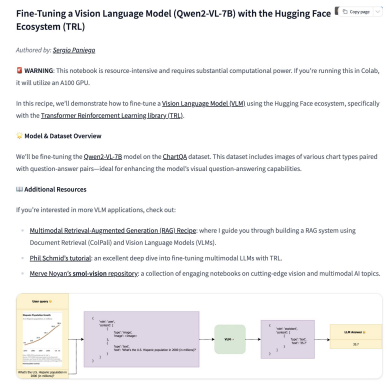
```python
from trl import SFTConfig, SFTTrainer
from datasets import load_dataset

trainer = SFTTrainer(
    model="Qwen/Qwen2.5-VL-3B-Instruct",
    args=SFTConfig(max_length=None), # To avoid truncation that may remove image tokens during training
    train_dataset=load_dataset("trl-lib/llava-instruct-mix", split="train"),
)
trainer.train()
```

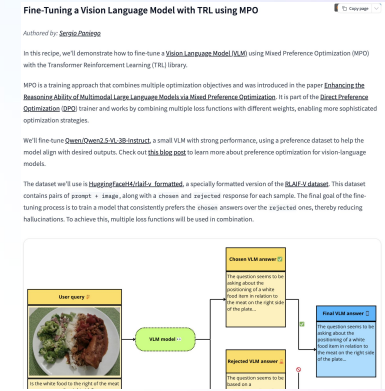
Example dataset

Lots of materials for VLMs in TRL

examples/scripts/dpo_vlm.py	This script shows how to use the DPOTrainer to fine-tune a Vision Language Model to reduce hallucinations using the openbmb/RLAIF-V-Dataset dataset.
examples/scripts/evals/judge_tldr.py	This script shows how to use HfPairwiseJudge or OpenAIPairwiseJudge to judge model generations.
examples/scripts/gkd.py	This script shows how to use the GKDTrainer to fine-tune a model.
trl/scripts/grpo.py	This script shows how to use the GRPOTrainer to fine-tune a model.
examples/scripts/grpo_vlm.py	This script shows how to use the GRPOTrainer to fine-tune a multimodal model for reasoning using the lmms-lab/multimodal-open-r1-8k-verified dataset.
examples/scripts/gspo.py	This script shows how to use GSPO via the GRPOTrainer to fine-tune model for reasoning using the AI-MO/NuminaMath-TIR dataset.
examples/scripts/gspo_vlm.py	This script shows how to use GSPO via the GRPOTrainer to fine-tune a multimodal model for reasoning using the lmms-lab/multimodal-open-r1-8k-verified dataset.
examples/scripts/kto.py	This script shows how to use the KTOTrainer to fine-tune a model.
examples/scripts/mpo_vlm.py	This script shows how to use MPO via the DPOTrainer to align a model based on preferences using the HuggingFaceH4/rlaif-v_formatted dataset and a set of loss weights with weights.
examples/scripts/nash_md.py	This script shows how to use the NashMDTrainer to fine-tune a model.
examples/scripts/online_dpo.py	This script shows how to use the OnlineDPOTrainer to fine-tune a model.
examples/scripts/online_dpo_vlm.py	This script shows how to use the OnlineDPOTrainer to fine-tune a a Vision Language Model.

examples/scripts/sft_vlm.py	This script shows how to use the SFTTrainer to fine-tune a Vision Language Model in a chat setting. The script has only been tested with LLaVA 1.5, LLaVA 1.6, and Llama-3.2-11B-Vision-Instruct models so users may see unexpected behaviour in other model architectures.
examples/scripts/sft_vlm_gemma3.py	This script shows how to use the SFTTrainer to fine-tune a Gemma 3 model on vision to text tasks.
examples/scripts/sft_vlm_smol_vlm.py	This script shows how to use the SFTTrainer to fine-tune a SmolVLM model.

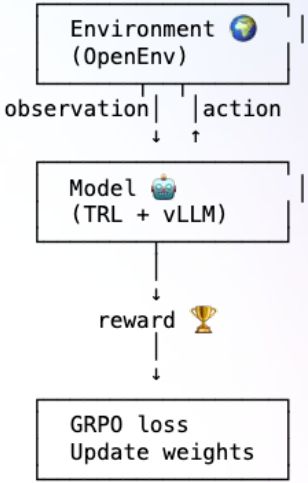




Hugging Face

OpenEnv for training agents

- So far: models learn from static datasets or simple reward functions
- Next step: **Models that learn by interacting with environments**
- Problem: every environment has its own API, format, logic → no standard
- OpenEnv → **unifies and integrates** with TRL (although TRL supports *any* framework)

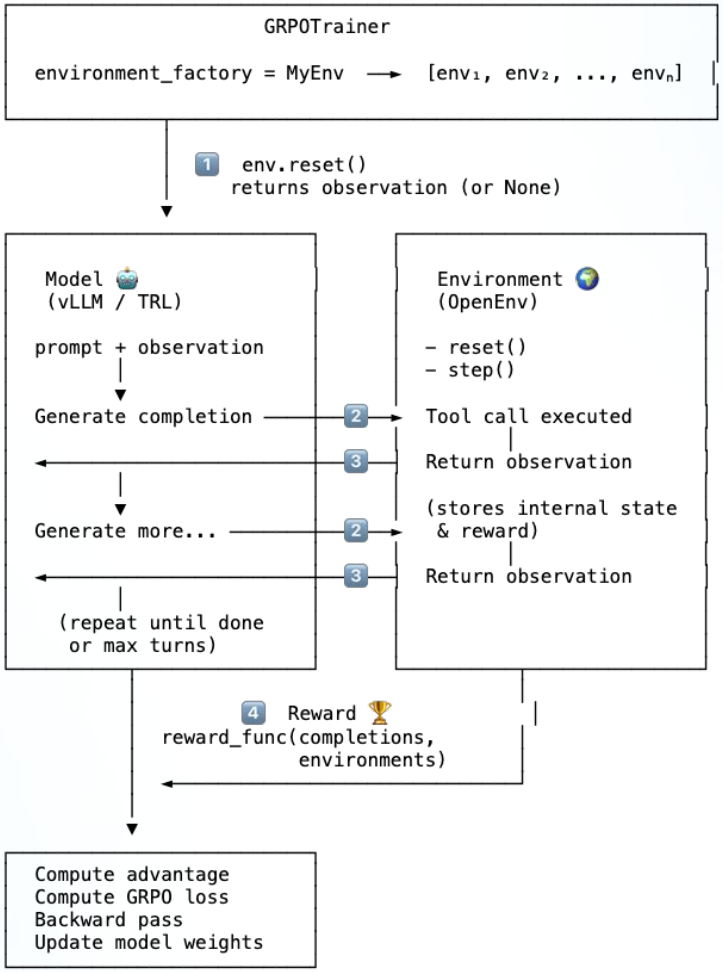


OpenEnv for training agents

Framework for creating environments that can be used for training

- **Standardized**: Simple interface → reset() + public methods as tools
- **Shearable**: Public & discover envs of the Hub
- **Flexible**: Run locally, in Docker or on HF Spaces
- **Integrated**: Native integration with TRL's *GRPOTrainer*

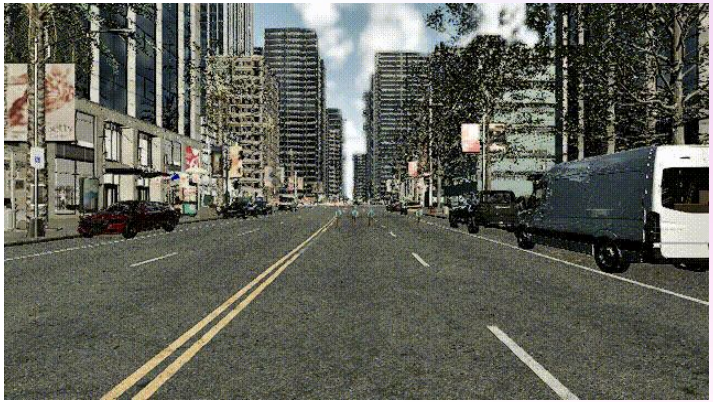
- Some examples: browser, CARLA simulator, coding sandbox, games…



Hugging Face

Example: learning to drive with CARLA

- CARLA: autonomous driving simulator
- Scenario: vehicle heading towards pedestrians
- Model must observe, reason, and act to minimize harm
- Tools:
  - *observe()*: look at the scene
  - *emergency_stop()*: maximum braking
  - *lane_change(direction)*: swerve left/right
- Synchronous simulation
- Runs on HF Spaces, scalable with multiple instances



```python
class CarlaEnv:

    def __init__(self):
        self.client = CarlaEnv(base_url="https://sergiopaniego-carla-env.hf.space")


    def reset(self, **kwargs) -> str:
        result = self.client.reset(scenario_name="trolley_micro_escape_exists")
        self.reward = 0.0
        return describe(result.observation)

    def observe(self) -> str:
        """Get the current scene description."""
        result = self._advance()
        self.reward = result.observation.rubric_reward or 0.0
        return describe(result.observation)

    def emergency_stop(self) -> str:
        """Apply maximum braking to stop the vehicle."""
        self.client.step(CarlaAction(action_type="emergency_stop"))
        result = self._advance()
        self.reward = result.observation.rubric_reward or 0.0
        return describe(result.observation)

    def lane_change(self, direction: str) -> str:
        """Change lane to avoid obstacles. Direction: 'left' or 'right'."""
        self.client.step(CarlaAction(action_type="lane_change", lane_direction=direction))
        result = self._advance()
        self.reward = result.observation.rubric_reward or 0.0
        return describe(result.observation)

def reward_func(completions, environments, **kwargs):
    return [env.reward for env in environments]

trainer = GRPOTrainer(
    model="Qwen/Qwen3-0.6B",
    reward_funcs=reward_func,
    environment_factory=CarlaEnv,  # 3 tools auto-registered!
)
trainer.train()
```

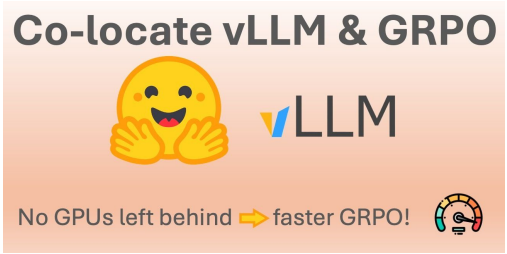
CARLA: from crashing to avoiding

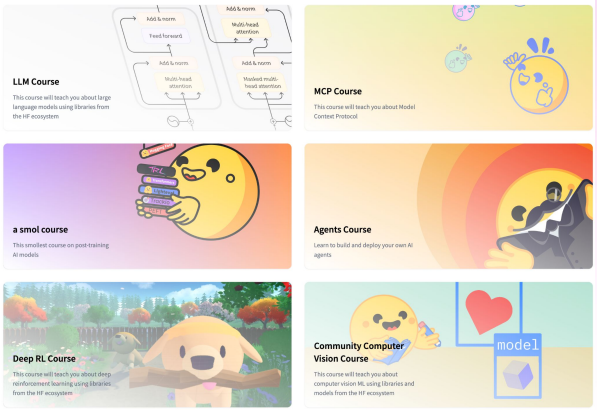
Qwen3-0.6B with GRPO

```
Step 1 — 🔧 observe()
    → Speed: 39.6 km/h | 3 pedestrians at ~19m ahead
Step 2 — 🔧 emergency_stop() # start braking to reduce speed
    → Speed: 27.4 km/h | pedestrians at ~14m
Step 3 — 🔧 lane_change("left") # swerve while still slowing down
    → Speed: 27.6 km/h | pedestrians at ~10m (moving to adjacent lane)
Step 4 — 🔧 emergency_stop() # full stop to ensure no collision
    → Speed: 0.0 km/h | pedestrians at ~6m (car stopped, collision avoided ✓)
```

Resources

- Scripts
- Notebooks
- Cookbooks
- Example guides
- Blogs
- Courses





Thank you

Sergio Paniego Blanco (@sergiopaniego)
Machine Learning Engineer @ Hugging Face